



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIEROS
INDUSTRIALES

MÁSTER EN ROBÓTICA Y AUTOMÁTICA
LABORATORIO DE ROBÓTICA Y AUTOMÁTICA

Clasificación de tapones plásticos (REC)

Autores:

Silvia Ochando Valero
Asil Arnous
Delia Martinez Fernandez
Ruth A. Bastidas Alva

Profesores:

Jaime Del Cerro



Madrid, 29 de mayo de 2026

ÍNDICE

1. Seguimiento del proyecto	2
1.1. Objetivos previstos	2
1.2. Objetivos alcanzados	3
1.3. Acciones correctivas	6
1.4. EDP e Hitos	8
1.5. Asignación de tareas y dedicación por sub-equipo	9
1.6. Diagrama de Gantt	11
2. Descripción del sistema desarrollado	12
2.1. Subsistema de Control	12
2.2. Subsistema de Visión	19
2.3. Interfaz Hombre-Máquina	24
2.4. Entorno de construcción y despliegue (Docker / ROS 2 Humble)	27
3. Pruebas de validación	28
3.1. Definición de las pruebas	28
3.2. Desarrollo de las pruebas y resultados	28
3.3. Análisis de resultados y validación de objetivos (requisitos)	29
4. Planos	31
4.1. Planos mecánicos	31
4.2. Planos electrónicos	31
5. Lista de materiales	31
6. Vídeos	31
7. ANEXO I: Especificación de Requisitos	31
8. ANEXO II	31

1. Seguimiento del proyecto

En esta tercera y última entrega se presenta el estado final del Proyecto REC: Clasificación de Tapones de Plásticos. A continuación se recogen los objetivos previstos para esta fase, los objetivos alcanzados, las acciones correctivas aplicadas durante el desarrollo y la planificación temporal definitiva del proyecto, incluyendo la EDP actualizada, los hitos finales y la asignación real de horas por participante y sub-equipo.

1.1. Objetivos previstos

Los objetivos planificados para esta tercera entrega correspondían a la finalización y validación integral del sistema, abarcando las fases de integración y validación (WP-500 y WP-600), la depuración final del sistema de visión (WP-300), la optimización de tiempos de ciclo en manipulación (WP-400) y el cierre documental y de gestión del proyecto (WP-100 y WP-200). A continuación se detallan los objetivos por paquete de trabajo:

WP-100 – Gestión del proyecto

- **103** – Especificación de requisitos del sistema: consolidación final del documento de requisitos funcionales y técnicos.
- **104** – Coordinación de actividades de los sub-equipos: coordinación final durante las fases de integración y validación.
- **107** – Control e integración de los cambios: cierre del repositorio con control de versiones y trazabilidad completa.

WP-200 – Diseño de la interfaz y arquitectura del sistema

- **201** – Diseño de la arquitectura de software: definición final de la estructura de nodos ROS2 (Humble) y de los tópicos de comunicación entre los paquetes `lra_vision`, `rec_vision`, `ur3_vision_control` y `lra_hmi`.
- **202** – Diseño de la interfaz de usuario: implementación de la HMI gráfica en PyQt5 (`lra_hmi`) como lanzador y panel de supervisión del sistema completo.
- **203** – Definición de protocolos de comunicación y lógica de control: establecimiento de las reglas de arranque, parada y parada de emergencia entre la HMI y el resto de nodos.

WP-300 – Sistema de Visión

- **308** – Depuración final del algoritmo: ajuste de parámetros de detección y mejora de la robustez del sistema de visión antes de la integración final.

WP-400 – Manipulación y Efector

- **405** – Optimización de tiempos de ciclo: ajuste final de aceleraciones, candidatos de orientación y aproximación para minimizar el tiempo de ejecución de cada ciclo de pick & place.

WP-500 – Integración

- **501** – Conexión de red y comunicación ROS2: puesta en marcha del driver del UR3e y del intercambio de mensajes entre los nodos de visión, control y HMI.
- **502** – Integración del sistema de visión con el sistema de manipulación: integración de coordenadas (/ur3/target_point) y caja asignada (/tapones/caja_asignada) con el nodo de manipulación del UR3e.
- **503** – Algoritmo de lógica de clasificación: clasificación de cada tapón por color mediante K-Means y asignación a una de las N cajas de descarga (configurable 1-6, por defecto 3).
- **504** – Sincronización y control de flujo: implementación del *gating* por /vision_enable para evitar lecturas de visión mientras el robot está en movimiento.
- **505** – Protocolo de gestión de excepciones: rutinas de seguridad para objetivos fuera de alcance, fallos de transformación TF y parada de emergencia desde la HMI.

WP-600 – Validación

- **601** – Definición de las pruebas de validación del sistema.
- **602** – Validación estadística de clasificación por color: ejecución de ciclos completos para verificar la robustez del algoritmo de clasificación.
- **603** – Verificación de seguridad y exclusión de colisiones: validación de que el planificador rechaza trayectorias que comprometan la integridad del entorno.
- **604** – Verificación de tiempos de ciclo: medición de tiempos finales de ejecución para comprobar el cumplimiento de las especificaciones.

1.2. Objetivos alcanzados

Al cierre del proyecto, la totalidad de los objetivos planificados han sido alcanzados. El sistema completo de clasificación de tapones plásticos está operativo, integrando los módulos de visión, manipulación, integración e interfaz de usuario sobre la arquitectura ROS2 Humble.

WP-100 – Gestión del proyecto

- **Tareas completadas: 101, 102, 103, 104, 105, 106, 107.**
 - La documentación final ha sido elaborada satisfactoriamente, incluyendo las tres entregas, los vídeos de seguimiento y el vídeo de demostración final.
 - El repositorio GitHub ha sido cerrado con control de versiones completo y los README de cada paquete actualizados.
 - Todas las reuniones de coordinación han sido realizadas y documentadas.

WP-200 – Diseño de la interfaz y arquitectura del sistema

- **Tareas completadas: 201, 202, 203.**

- La arquitectura se ha estructurado en cuatro paquetes ROS2 con responsabilidades separadas: `lra_vision` (cámara y calibración), `rec_vision` (detección y clasificación), `ur3_vision_control` (manipulación) y `lra_hmi` (interfaz de usuario).
- La Interfaz Gráfica de Usuario (HMI) en PyQt5 ha sido implementada como lanzador único del sistema, sustituyendo el arranque manual por terminales. Permite iniciar, detener y reiniciar cada subsistema, dispone de parada de emergencia, monitor de conexión del UR3e, contadores por caja y totales, lectura de ángulos articulares y visualización de la cámara en directo.
- Los protocolos de comunicación entre la HMI y el resto de nodos han quedado definidos y validados.

WP-300 – Sistema de Visión

- **Tareas completadas: 301, 302, 303, 304, 305, 306, 307, 308.**

- El algoritmo de visión ha sido depurado y optimizado en su versión final. La detección de tapones mediante la Transformada de Hough Circular sobre la ROI de la caja de trabajo (segmentada en HSV) funciona de forma robusta y estable.
- La clasificación por color se realiza con K-Means: un nodo calibrador (`color_calibrator_node`) se ejecuta una sola vez al inicio, agrupa los tapones y publica de forma persistente los centros HSV de cada caja, que el nodo detector consume para asignar cada tapón.
- El número de cajas es configurable (de 1 a 6, por defecto 3) y se fija desde la interfaz de usuario antes de arrancar la clasificación.
- El mecanismo de selección estable por análisis multifotograma (30 frames con *clustering* espacial) minimiza los falsos positivos, y se ha fijado una semilla aleatoria para garantizar la reproducibilidad de la detección y la clasificación entre arranques.
- La transformación de coordenadas píxel a espacio 3D del robot se realiza con el modelo *pinhole* (corrección de distorsión con `undistortPoints`) y la intersección del rayo con el plano de la mesa mediante el árbol de transformaciones TF2.

WP-400 – Manipulación y Efector

- **Tareas completadas: 401, 402, 403, 404, 405.**

- El efector final validado es una ventosa, montada como herramienta del TCP del UR3e. La detección de visión ya no calcula el diámetro del tapón, al no requerirse para la recogida por vacío.
- La secuencia de pick & place (*Home* → *Aproximación* → *Pick* → *Retreat* → *Clasificación* → *Drop* → *Home*) funciona de forma fiable y continua para todas las cajas configuradas.

- La optimización de tiempos de ciclo se ha apoyado en candidatos de orientación, aproximación, contacto y retirada, que permiten al planificador encontrar soluciones válidas reduciendo los intentos de movimiento inviables.
- El control de la ventosa mediante el servicio `/io_and_status_controller/set_io` y la resolución de cinemática inversa mediante `/compute_ik` operan correctamente, con un modo de simulación de efector para pruebas sin hardware.

WP-500 – Integración

- **Tareas completadas: 501, 502, 503, 504, 505.**

- La comunicación entre los nodos de visión, el controlador del UR3e y la HMI ha sido establecida y validada con éxito.
- La integración visión–manipulación funciona de forma cohesionada: el nodo de visión publica la posición del tapón (`/ur3/target_point`) y la caja asignada (`/tapones/caja_asignada`), y el nodo de manipulación ejecuta la maniobra correspondiente.
- El *gating* por `/vision_enable` (publicado de forma *latched* por el nodo de manipulación) garantiza que no se envíen nuevos objetivos de visión mientras el robot está ejecutando una maniobra; cada flanco de subida produce una única detección nueva.
- El protocolo de excepciones opera correctamente: los objetivos no clasificables o fuera del alcance del robot se descartan de forma segura, los fallos de transformación TF re-arman el muestreo, y la parada de emergencia de la HMI desactiva la visión y detiene el sistema.

WP-600 – Validación

- **Tareas completadas: 601, 602, 603, 604.**

- La validación estadística sobre ciclos completos de clasificación ha confirmado una tasa de acierto elevada, cumpliendo el requisito funcional RF-01.
- La verificación de seguridad ha confirmado que el sistema rechaza trayectorias fuera del espacio de trabajo y que el retorno a *Home* no invade el espacio de las cajas.
- Los tiempos de ciclo finales medidos cumplen las especificaciones del proyecto, priorizando la seguridad y la repetibilidad sobre la velocidad máxima de operación.

La siguiente tabla resume el estado final de los WP del proyecto:

Tabla 1: Planificación temporal general del proyecto.

WP	Tareas totales	Terminado	En progreso	% Completado
Gestión	7	7	0	100 %
Diseño de la interfaz y arquitectura	3	3	0	100 %
Sistema de visión	8	8	0	100 %
Manipulación y efector	5	5	0	100 %
Integración	5	5	0	100 %
Validación	4	4	0	100 %

1.3. Acciones correctivas

Durante la fase final del proyecto se han identificado y gestionado las siguientes incidencias. Ninguna de ellas ha comprometido los hitos críticos del proyecto.

WP-300 – Sistema de Visión

- **Incidencia:** La clasificación de color asignaba cajas erróneas de forma intermitente. El análisis del código reveló que la causa no era la extracción del color, sino la forma de tratar el matiz: la media aritmética del canal H es incorrecta porque H es circular (en OpenCV, 0–180), de modo que un tapón con píxeles cerca de ambos extremos (p. ej. rojo en $H \approx 175$ y $H \approx 5$) producía una media $H \approx 90$, interpretada como un color distinto. A ello se sumaban los reflejos especulares del plástico (brillo de V alta y S baja que contamina la media) y una distancia de color basada en la norma euclídea sobre [H,S,V] crudos, en la que S y V dominaban sobre H por su mayor escala.
 - **Acción correctiva:** Se reescribió la extracción de color (idéntica en el nodo calibrador y en el detector, ya que ambos deben medir igual): se usa un anillo interior ($0,25r$ a $0,75r$) en lugar del círculo completo para evitar el reflejo central y el borde con sombras, se descartan los píxeles de reflejo especular y de sombra, se toma la *mediana* de S y V (robusta a valores atípicos) y la *media circular* de H (mediante seno y coseno), resolviendo el problema del rojo a caballo entre ambos extremos. La distancia de clasificación se sustituyó por una métrica que respeta la circularidad de H, normaliza las tres componentes a 0–1 y *pondera H por la saturación*, de forma que en colores casi grises (negro/blanco), donde el matiz es ruido, la decisión recae sobre S y V.

WP-400 / WP-500 – Manipulación e Integración

- **Incidencia:** En ocasiones el robot no depositaba el tapón en la caja, llevándose en el movimiento de retorno. El análisis del ciclo mostró que tras la apertura del efector (`open_gripper()`) en la posición de descarga no había una espera, de modo que el robot iniciaba el siguiente movimiento antes de que la ventosa hubiese soltado realmente la pieza. Análogamente, la espera tras el cierre del efector era demasiado corta para garantizar un agarre firme, especialmente con el efector real.
 - **Acción correctiva:** Se introdujo una espera tras la apertura del efector en la descarga y se amplió la espera tras el cierre, asegurando que la activación y desactivación de vacío se completen antes de continuar la trayectoria.
- **Incidencia:** El nodo de manipulación acumulaba objetivos de tapones anteriores y, ante un error o al finalizar un ciclo, conservaba el último objetivo y color sin limpiarlos, lo que provocaba que se intentara procesar de nuevo una pieza ya gestionada.
 - **Acción correctiva:** Se redujo la profundidad de cola a 1 (`KEEP_LAST=1`) en las suscripciones de posición y caja, de modo que solo se conserva el último mensaje, y se añadió un reinicio explícito del estado interno (`last_target`, `last_color`, `current_target`) en todas las rutas de salida, incluidas las rutas de error y de caja no reconocida.
- **Incidencia:** En los registros se observaban *timeouts* esperando la respuesta del servicio de cinemática inversa (`/compute_ik`) y en algunos movimientos hacia la caja, al estar los tiempos límite ajustados al borde de lo necesario.
 - **Acción correctiva:** Se ampliaron los *timeouts* críticos del ciclo: el de `/compute_ik` se elevó a 2,0 s y se holgaron los tiempos de aproximación, descarga, movimiento y retirada hacia las cajas, eliminando los fallos por tiempo agotado.
- **Incidencia:** El planificador rechazaba algunas poses de recogida al no encontrar solución de cinemática inversa con una orientación y aproximación fijas.
 - **Acción correctiva:** Se introdujeron listas de candidatos de orientación, aproximación, contacto y retirada, de modo que el nodo prueba varias alternativas antes de descartar un objetivo, aumentando la tasa de poses planificables.

1.4. EDP e Hitos

EDP – Estado Final

La Estructura de Desglose del Proyecto (EDP) mantiene los seis paquetes funcionales definidos en la Entrega 2. Todos han alcanzado el 100 % de completitud al cierre del proyecto. No se han introducido modificaciones estructurales respecto a la Entrega 2; las tareas previstas para esta fase han sido ejecutadas según lo planificado, con los ajustes menores recogidos en las acciones correctivas anteriores.

Hitos – Estado Final

Tabla 2: Hitos del proyecto – Estado final

ID	Hito	Descripción	Fecha	Estado
H1	Planificación finalizada	Alcance, EDP, planificación temporal y requisitos definidos.	06/03/2026	✓ OK
H2	Visión preliminar	Algoritmo de detección implementado y probado con imágenes estáticas.	27/03/2026	✓ OK
H3	Manipulación operativa	Ciclo pick & place funcional con trayectorias de seguridad y retorno a Home.	27/03/2026	✓ OK
H4	Visión en tiempo real	Algoritmo validado con la cámara en condiciones reales del laboratorio.	12/04/2026	✓ OK
H5	Manipulación optimizada	Trayectorias y tiempos de ciclo del UR3e ajustados y validados.	17/04/2026	✓ OK
H6	Integración completada	Comunicación estable visión–robot e interfaz de usuario operativa.	08/05/2026	✓ OK
H7	Validación del sistema	Pruebas de clasificación continua y verificación de protocolos de seguridad.	19/05/2026	✓ OK
H8	Sistema final operativo	Sistema completamente funcional y listo para la demostración final.	22/05/2026	✓ OK

Todos los hitos han sido alcanzados dentro de los plazos establecidos. Los hitos H4 y H5, desplazados al 12/04/2026 por el retraso parcial en la tarea 304 (documentado en la Entrega 2), fueron completados en la fecha ajustada sin afectar al inicio del WP-500.

1.5. Asignación de tareas y dedicación por sub-equipo

A continuación se presentan las tablas de asignación de horas reales dedicadas por cada participante en cada tarea del proyecto. El equipo se organiza en dos sub-equipos: **Visión** (Delia Martínez – DM, y Asil Arnous – AA) y **Manipulación** (Silvia Ochoa – SO, y Ruth Bastidas – RB).

Los paquetes de gestión (WP-100), interfaz y arquitectura (WP-200), integración (WP-500) y validación (WP-600) se han abordado de forma conjunta entre los cuatro integrantes, mientras que el sistema de visión (WP-300) corresponde al sub-equipo de Visión y la manipulación y el efector (WP-400) al sub-equipo de Manipulación. Los valores reflejan la dedicación acumulada a lo largo de todo el proyecto, incluyendo reuniones, desarrollo técnico, pruebas y documentación.

Gestión, interfaz y arquitectura (WP-100 y WP-200)

Tabla 3: Asignación de horas – Gestión, Interfaz y Arquitectura

Tarea	DM	AA	SO	RB	Total
101 – Gestión de reuniones	6	6	6	6	24
102 – Planificación del proyecto	6	6	6	6	24
103 – Especificación de requisitos	6	6	6	6	24
104 – Coordinación sub-equipos	6	6	6	6	24
105 – Documentación del proyecto	7	7	7	7	28
106 – Vídeos de avance	6	6	6	6	24
107 – Control e integración cambios	5	5	5	5	20
201 – Arquitectura de software	6	6	6	6	24
202 – Interfaz de usuario (HMI)	6	6	6	6	24
203 – Protocolos de comunicación	6	6	6	6	24
Total horas	60	60	60	60	240

Sub-equipo de Manipulación — WP-400 (SO y RB)

Tabla 4: Asignación de horas – Manipulación y Efector

Tarea	SO	RB	Total
401 – Análisis de geometría y agarre	7	7	14
402 – Trayectorias de seguridad	8	8	16
403 – Configuración de TCP (ventosa)	7	7	14
404 – Ciclo de Pick & Place	10	10	20
405 – Optimización de tiempos de ciclo	8	8	16
Total horas	40	40	80

Sub-equipo de Visión — WP-300 (DM y AA)

Tabla 5: Asignación de horas – Sistema de Visión

Tarea	DM	AA	Total
301 – Arquitectura HW/SW sistema de visión	7	7	14
302 – Selección de software de visión	7	7	14
303 – Selección de algoritmos de visión	7	8	15
304 – Elaboración del algoritmo de visión	9	9	18
305 – Calibración de la cámara	7	7	14
306 – Pruebas con imágenes grabadas	7	7	14
307 – Integración con cámara en tiempo real	8	8	16
308 – Depuración final del algoritmo	8	7	15
Total horas	60	60	120

Integración y validación (WP-500 y WP-600)

Tabla 6: Asignación de horas – Integración y Validación

Tarea	DM	AA	SO	RB	Total
501 – Conexión de red y ROS2	5	5	5	5	20
502 – Integración visión–manipulación	6	6	6	6	24
503 – Algoritmo de clasificación	5	5	5	5	20
504 – Sincronización y control de flujo	5	5	5	5	20
505 – Protocolo de excepciones	5	5	5	5	20
601 – Definición de pruebas de validación	4	4	4	4	16
602 – Validación estadística por color	5	5	5	5	20
603 – Verificación de seguridad	5	5	5	5	20
604 – Verificación de tiempos de ciclo	4	4	4	4	16
Total horas	44	44	44	44	176

1.6. Diagrama de Gantt

El diagrama de Gantt definitivo refleja la planificación temporal real ejecutada a lo largo del proyecto. Las únicas desviaciones respecto al cronograma de la Entrega 2 son el desplazamiento de las tareas 304, 306 y 307 del WP-300 hasta el 12/04/2026 (justificado en la Entrega 2) y la incorporación de las tareas del WP-200 a partir del 08/04/2026. El resto del cronograma se ha ejecutado sin modificaciones.

[Insertar diagrama de Gantt definitivo aquí]

Figura 1. Diagrama de Gantt definitivo del Proyecto REC.

2. Descripción del sistema desarrollado

El sistema desarrollado clasifica tapones de plástico por color empleando un robot UR3e dotado de un efector de vacío y un sistema de visión por computador, integrados sobre ROS 2 Humble. La arquitectura se organiza en cuatro paquetes con responsabilidades separadas: `lra_vision` (infraestructura de cámara, calibración y modelo cinemático cámara-robot), `rec_vision` (detección y clasificación de tapones), `ur3_vision_control` (control del robot y ciclo de pick & place) y `lra_hmi` (interfaz de supervisión y lanzador). En esta sección se describe cada subsistema, sus interfaces y la algorítmica empleada.

2.1. Subsistema de Control

2.1.1. Visión general

El subsistema de control tiene como objetivo gobernar el robot UR3e para ejecutar ciclos de recogida y clasificación de tapones de colores sobre una mesa de trabajo. El nodo principal, denominado `ur3_pick_sort`, integra la cinemática inversa (IK) mediante MoveIt 2, la gestión de la pinza por vacío, el control de la visión y la publicación de objetos de colisión en la escena de planificación.

El ciclo de operación completo es el siguiente: el sistema de visión detecta un tapón y publica su posición en el espacio cartesiano junto con su color; el nodo de control recibe ambos mensajes, resuelve la trayectoria articular mediante IK, ejecuta las etapas de aproximación, agarre, retirada y depósito en la caja correspondiente, y finalmente reactiva la visión para el siguiente ciclo.

2.1.2. Arquitectura software

Stack tecnológico. La Tabla 7 resume las tecnologías empleadas en el subsistema de control.

Tabla 7: Stack tecnológico del subsistema de control.

Componente	Tecnología / Versión	Rol
Sistema operativo	Ubuntu 22.04 LTS	Plataforma base
Middleware robótico	ROS 2 Humble	Comunicación inter-proceso
Planificador de movimiento	MoveIt 2 (Humble)	Planificación de trayectorias e IK
Controlador de articulaciones	<code>ros2_control + scaled_joint_trajectory_controller</code>	Ejecución de trayectorias en hardware
Driver del robot	<code>ur_robot_driver</code>	Interfaz con el UR3e real
Lenguaje de nodos	Python 3.10	Lógica de control
Launch system	Python Launch Files (ROS 2)	Orquestación del arranque
Visualización	RViz 2 (MarkerArray)	Depuración visual en tiempo real

Nodos ROS 2 del subsistema de control.

- `ur3_pick_sort` – Nodo principal. Implementa toda la lógica de pick-and-place, IK, gestión de la pinza y coordinación con el sistema de visión (paquete `ur3_vision_control`).
- `ur3_moveit_test` – Nodo de prueba de integración con Moveit 2 a través del *action server* `/move_action`. Útil para validar la configuración del stack de forma aislada.
- `ur3_simple_move` (`inicio.py`) – Script de utilidad para llevar el robot a una pose articular fija; se usa en la puesta en marcha y la calibración de la posición HOME.
- `ur3_controller` (`ur3_controller.py`) – Prototipo anterior de control directo por ángulos heurísticos; no se usa en producción, sirve como referencia de desarrollo.

Diagrama de nodos y tópicos. La Tabla 8 describe los tópicos principales con su dirección de flujo respecto al nodo `ur3_pick_sort`.

Tabla 8: Tópicos y servicios del subsistema de control.

Tópico	Tipo	Dir.	Descripción
<code>/ur3/target_point</code>	<code>geometry_msgs/Point</code>	Ent.	Posición XYZ del tapón en <code>base_link</code>
<code>/tapones/caja_asignada</code>	<code>std_msgs/Int32</code>	Ent.	Índice de color/caja (1–6)
<code>/joint_states</code>	<code>sensor_msgs/JointState</code>	Ent.	Estado articular (500 Hz)
<code>/vision_enable</code>	<code>std_msgs/Bool</code>	Sal.	Habilita el detector (latched)
<code>/scaled_joint_traj.../joint_trajectory</code>	<code>trajectory_msgs/JointTrajectory</code>	Sal.	Trayectoria al controlador HW
<code>/io_and_status_controller/set_io</code>	<code>ur_msgs/SetIO (srv)</code>	Sal.	Activa/desactiva la ventosa
<code>/compute_ik</code>	<code>moveit_msgs/GetPositionIK (srv)</code>	Bidir.	Solicitud de IK a Moveit 2
<code>/collision_object</code>	<code>moveit_msgs/CollisionObject</code>	Sal.	Objetos de colisión (mesa, pared)
<code>/sorting_markers</code>	<code>visualization_msgs/MarkerArray</code>	Sal.	Marcadores RViz (HOME, cajas, target)
<code>/tf</code>	<code>tf2_msgs/TFMessage</code>	Ent.	TF <code>base_link</code> → <code>tool0</code>

2.1.3. Interfaces entre procesos

Interfaz con el subsistema de visión. La comunicación entre visión y control se realiza exclusivamente mediante tópicos ROS 2, garantizando desacoplamiento temporal y espacial entre ambos sub-equipos:

- **Posición** (`/ur3/target_point`, `geometry_msgs/Point`): campos x, y, z en metros, referenciados a `base_link`. QoS *best-effort*, cola de profundidad 1: el controlador procesa únicamente el último *target*, descartando detecciones acumuladas durante el ciclo de pick. El nodo de visión publica solo cuando `/vision_enable=True`.
- **Clasificación** (`/tapones/caja_asignada`, `std_msgs/Int32`): codificación 1–6 = {rojo, azul, verde, amarillo, negro, blanco}. El controlador acepta también cadenas de texto vía `color_callback` (nombres en español a índices). Cola de profundidad 1.

- **Habilitación** (`/vision_enable`, `std_msgs/Bool`): QoS RELIABLE + TRANSIENT_LOCAL (*latched*). El uso de TRANSIENT_LOCAL garantiza que cualquier nodo de visión que arranque después del controlador reciba inmediatamente el último estado, evitando que el detector comience a procesar antes de que el robot haya alcanzado HOME.

Interfaz con el hardware del robot (`ros2_control`). El UR3e se controla a través del `scaled_joint_trajectory_controller`, que escala la velocidad de ejecución respecto a la máxima configurada en la *teach pendant*. El nodo publica mensajes `JointTrajectory` con un único punto de destino y un tiempo de llegada explícito (`time_from_start`); la interpolación la realiza internamente el controlador hardware.

Interfaz con MoveIt 2 (IK). La cinemática inversa se resuelve llamando al servicio `/compute_ik` (`moveit_msgs/GetPositionIK`) de forma asíncrona mediante `call_async()` con un `threading.Event` para el bloqueo, ya que el ciclo de pick-and-place corre en un hilo secundario. Parámetros relevantes de la petición: `group_name=ur_manipulator`, `ik_link_name=tool0`, `pose_stamped` en `base_link`, `robot_state` con semilla articular fija (HOME) para reproducibilidad, `timeout` de 2 s y `avoid_collisions=True` con *fallback* a False si no se encuentra solución.

Interfaz con la pinza de vacío (`SetIO`). La ventosa se controla mediante `/io_and_status_controller/set_io` (`ur_msgs/SetIO`): función=1 (salida digital), `pin=4`, `estado=1.0` (cierre) o 0.0 (apertura). El parámetro `simulate_gripper` sustituye la llamada real por un *log*, útil para desarrollo sin hardware.

2.1.4. Interfaz de usuario

El subsistema de control no dispone de interfaz gráfica propia. La interacción con el operador se realiza mediante tres mecanismos:

- **Parámetros de lanzamiento** (Tabla 9), configurables desde el archivo de *launch* o la línea de comandos.
- **Visualización en RViz 2:** el nodo publica un `MarkerArray` en `/sorting_markers` a 2 Hz con una esfera blanca (TCP en HOME), cubos de color (las 6 cajas de destino con etiquetas) y una esfera magenta (target activo durante el ciclo).
- **Logs de consola:** el nodo emite logs en cada transición de estado relevante (recepción de target, inicio de ciclo, etapas de IK, movimientos, estado de la pinza, tiempos y reactivación de visión), permitiendo seguir el sistema sin herramientas externas.

Tabla 9: Parámetros ROS 2 del nodo `ur3_pick_sort`.

Parámetro	Tipo	Defecto	Descripción
<code>simulate_gripper</code>	bool	True	Sustituye la pinza por logs (modo simulación)
<code>offset_x</code>	float	0.0	Corrección métrica en X del target
<code>offset_y</code>	float	0.0	Corrección métrica en Y del target

Scripts de arranque. El archivo `launch.py` orquesta el arranque completo con temporizaciones explícitas: $t = 0$ s (MoveIt 2 + cámara + URDF de cámara + calibrador de color), $t = 8$ s (nodo `ur3_pick_sort`, tras estabilización de MoveIt) y $t = 25$ s (detector de tapones, tras llegada del robot a HOME). El archivo `launch_vision.sh` permite arrancar solo el *pipeline* de visión, útil para calibración y depuración sin el robot activo.

2.1.5. Algorítmica del sistema de control

Máquina de estados implícita. El nodo implementa una máquina de estados mediante el flag booleano `busy` y temporizadores ROS 2 (Tabla 10).

Tabla 10: Estados del nodo de control.

Estado	Entrada	Acciones	Salida
INICIALIZANDO	Arranque del nodo	Deshabilita visión, abre pinza, inicia temporizadores de HOME	Robot en HOME
EN HOME / ESPERANDO	Robot en HOME	Habilita visión, <code>busy=False</code>	Recepción de target + color
EJECUTANDO CICLO	<code>try_start_cycle()</code> OK	<code>busy=True</code> , deshabilita visión, lanza hilo de pick-and-place	Fin de ciclo
ABORTANDO	Error en IK o movimiento	Retorna a HOME, limpia estado, reactiva visión	Robot en HOME

Ciclo de pick-and-place. El ciclo completo se ejecuta en un hilo secundario (*daemon thread*) para no bloquear el *executor* de ROS 2, que debe seguir procesando *callbacks* de `joint_states` y respuestas IK. Las etapas son:

1. **Validación del workspace:** se comprueba que (x, y, z) estén en $x \in [-0,60, 0,80]$, $y \in [-0,30, 0,55]$, $z \in [0,0001, 0,30]$ m.
2. **IK de APPROACH:** TCP a ~ 22 cm sobre el tapón, con semilla fija en HOME.
3. **Movimiento bloqueante a APPROACH:** confirmación por `/joint_states` (tol. 0.03 rad).
4. **IK de PICK:** contacto con el tapón (22 cm – compresión).
5. **Movimiento a PICK + cierre de ventosa:** descenso y activación de vacío; espera de 0.7 s.
6. **Movimiento a HOME intermedio:** el robot va directamente a HOME con el tapón.
7. **Movimiento a BIN APPROACH:** pose articular pregrabada para el color detectado.
8. **Movimiento a BIN DROP:** pose de depósito (pequeña variación angular desde APPROACH).
9. **Apertura de ventosa + espera 0.8 s:** se suelta el tapón.
10. **Retorno a BIN APPROACH** (si las poses difieren) y **HOME final**.
11. **Limpieza de estado y reactivación de visión.**

Detección de llegada a waypoint. La función `move_to_joint_waypoint_blocking` lee `/joint_states` a 10 Hz y compara la posición articular con el objetivo mediante la distancia angular circular:

$$d(a, b) = |((a - b + \pi) \bmod 2\pi) - \pi|. \quad (1)$$

El waypoint se considera alcanzado cuando todos los joints cumplen $d < 0,03$ rad ($\approx 1,7^\circ$), con un *timeout* configurable por etapa (entre 8 y 45 s). Si expira, el sistema entra en modo aborto. Una optimización verifica si el robot ya está en el waypoint antes de publicar la trayectoria, evitando movimientos redundantes.

Normalización de ángulos. Todos los ángulos se normalizan al intervalo $(-\pi, \pi]$ mediante `normalize(q) = atan2(sin q, cos q)`. Para evitar giros innecesarios de 360° al transitar entre poses se aplica

$$\text{wrap_to_near}(q_{\text{ref}}, q_{\text{tgt}}) = q_{\text{ref}} + ((q_{\text{tgt}} - q_{\text{ref}} + \pi) \bmod 2\pi - \pi), \quad (2)$$

garantizando que el controlador recibe siempre la solución articular más próxima a la posición actual.

2.1.6. Matemáticas: cinemática inversa y selección de solución

Planteamiento. Dado un punto objetivo $P = (x, y, z)$ en `base_link`, la IK busca el vector articular $q = (q_1, \dots, q_6)$ tal que la pose del TCP (`tool0`) coincida con la deseada: $T(q) = T_{\text{base_link} \rightarrow \text{tool0}}(q_1, \dots, q_6)$, donde T es la transformada homogénea 4×4 de la cadena cinemática del UR3e según la convención Denavit–Hartenberg modificada. El solver es KDL (*Kinematics and Dynamics Library*), integrado en MoveIt 2.

Orientación del TCP. Para el pick se requiere el TCP apuntando hacia abajo (perpendicular a la mesa). La orientación base es una rotación de π alrededor de X en el efector, $R_{\text{base}} = \text{Euler}(\pi, 0, 0)$, es decir $q = (1, 0, 0, 0)$. Para robustez, se prueban 4 orientaciones variando el *yaw*: $\{0, +\pi/2, -\pi/2, \pi\}$, ya que la orientación del tapón sobre la mesa es irrelevante para el agarre.

Conversión Euler $\rightarrow$ cuaternión. La conversión (ZYX) se implementa directamente, sin dependencias externas, con $c_\bullet = \cos(\bullet/2)$, $s_\bullet = \sin(\bullet/2)$:

$$q_x = s_r c_p c_y - c_r s_p s_y, \quad q_y = c_r s_p c_y + s_r c_p s_y, \quad (3)$$

$$q_z = c_r c_p s_y - s_r s_p c_y, \quad q_w = c_r c_p c_y + s_r s_p s_y. \quad (4)$$

Búsqueda multi-candidato (`solve_stage_ik`). La función explora un espacio de candidatos para maximizar la probabilidad de hallar una solución IK válida y de bajo coste (Tabla 11). El espacio total es de hasta $3 \times 3 \times 4 \times 2 = 72$ llamadas IK por etapa; en la práctica la primera semilla (HOME) converge en las primeras iteraciones, por lo que el tiempo real es muy inferior al peor caso.

Tabla 11: Dimensiones de búsqueda de `solve_stage_ik`.

Dimensión	Cand.	Descripción
Semillas articulares	3	HOME (fija), posición actual, HOME (respaldo)
Z del objetivo	3	z_{nom} , $z_{\text{nom}} \pm 0,02$ (approach) o $\pm 0,005$ (pick)
Orientación TCP (yaw)	4	$0, +\pi/2, -\pi/2, \pi$
Modo colisión	2	<code>avoid_collisions=True</code> (primero), <code>False</code> (fallback)

Función de coste articular. Entre todas las soluciones IK válidas se elige la de menor coste, que combina el coste de movimiento y la penalización por alejarse de la rama preferida de pick:

$$\text{coste}(q_{\text{ref}}, q_{\text{tgt}}) = \text{coste}_{\text{mov}} + \text{coste}_{\text{rama}}, \quad (5)$$

$$\text{coste}_{\text{mov}} = \sum_i w_i d_{\text{ang}}(q_{\text{ref},i}, q_{\text{tgt},i}), \quad (6)$$

$$\text{coste}_{\text{rama}} = \lambda \sum_i w_i d_{\text{ang}}(q_{\text{tgt},i}, q_{\text{home},i}), \quad (7)$$

con pesos $w = [1,0, 4,0, 3,5, 2,0, 2,0, 1,0]$ (mayor peso a *shoulder_lift* y *elbow*, más influyentes en la postura), $\lambda = 0,20$ y d_{ang} la distancia circular mínima entre dos ángulos. La semilla fija (HOME) garantiza que, ante el mismo target (x, y) , el solver devuelve siempre la misma solución entre arranques.

Geometría TCP y offsets de pick. El efector final es un *gripper* neumático más una ventosa, con longitud total de 230 mm ($L_{\text{gripper}} = 137$ mm, $L_{\text{ventosa}} = 93$ mm). Los offsets Z aplicados a la posición del tapón se recogen en la Tabla 12. El offset de compresión de 5 mm asegura el contacto firme de la ventosa, necesario para generar el vacío de agarre.

 Tabla 12: Offsets Z por etapa ($L_{\text{TCP}} = 0,230$ m).

Etapas	Offset Z (m)	Descripción
APPROACH	$L_{\text{TCP}} + 0,08 = 0,310$	TCP a 8 cm sobre el tapón
PICK (contacto)	$L_{\text{TCP}} - 0,005 = 0,225$	5 mm de compresión
RETREAT (en HOME)	$L_{\text{TCP}} + 0,12 = 0,350$	Elevación de seguridad

2.1.7. Gestión de la escena de colisión en Movelt 2

Para que el solver IK con `avoid_collisions=True` funcione, el nodo publica los objetos de colisión del entorno físico en `/collision_object` al arrancar (Tabla 13). La mesa está rotada 45° alrededor de Z (cuaternión $[-0,707, 0, 0, 0,707]$) para alinearla con la disposición física. Estos objetos se eliminan y re-publican en cada arranque para evitar duplicados en la escena persistente de Movelt.

Tabla 13: Objetos de colisión publicados en la escena.

Objeto	Geom.	Posición (world)	Dimensiones
table	BOX	$x=0,20, y=-0,10, z=-0,02$	$1,40 \times 0,90 \times 0,04$ m
back_wall	BOX	$x=-0,25, y=0,00, z=0,30$	$0,06 \times 0,90 \times 0,60$ m

2.1.8. Gestión del threading y concurrencia

El nodo opera con un único *executor* de ROS 2 (`rclpy.spin`) en el hilo principal. Para que los ciclos de pick-and-place (con bucles de espera activa y llamadas bloqueantes) no impidan el procesamiento de *callbacks*, se separan en un hilo secundario:

```
threading.Thread(target=self.execute_pick_and_place,
                 args=(target, color), daemon=True).start()
```

El hilo secundario realiza llamadas asíncronas a los servicios (`compute_ik`, `set_io`) usando `threading.Event` para sincronizar y evitar *deadlocks* con el *executor*:

```
future = self.compute_ik_client.call_async(req)
done_event = threading.Event()
future.add_done_callback(lambda fut: done_event.set())
done_event.wait(timeout=timeout_sec + 1.0)
```

El flag `busy` actúa como *mutex* ligero: solo un ciclo está activo a la vez. Las suscripciones a `/ur3/target_point` y `/tapones/caja_asignada` retornan inmediatamente si `busy=True`, sin encolar peticiones.

2.1.9. Configuración de waypoints de destino (bins)

Las posiciones de depósito de cada color están pregrabadas como vectores articulares en grados, convertidos a radianes y normalizados al intervalo $(-\pi, \pi]$ en el arranque (Tabla 14). Para cada color se definen dos waypoints (APPROACH y DROP); la pose de DROP se obtiene aplicando un delta articular pequeño sobre APPROACH (principalmente *elbow* y *wrist_1*), produciendo un descenso hacia el interior de la caja sin recalculer IK. Si APPROACH y DROP son prácticamente iguales ($d < 0,03$ rad en todos los joints), se omite el retorno a APPROACH tras el depósito.

Tabla 14: Poses articulares de approach por color/caja (grados).

Color	Caja	q (approach)
Rojo	1	[-252, -108, -68, -96, 87, -60]
Azul	2	[-236, -114, -56, -99, 90, -50]
Verde	3	[-223, -139, -14, -118, 90, -35]
Amarillo	4	[-241, -70, -104, -96, 88, -30]
Negro	5	[-216, -80, -98, -94, 90, -30]
Blanco	6	[-204, -97, -77, -94, 90, -30]

2.1.10. Relación con el subsistema de visión

El subsistema de control opera de forma completamente reactiva: no realiza procesamiento de imagen ni decide sobre la identidad o posición de los tapones. Toda esa responsabilidad recae en el subsistema de visión (Sección 2.2.2). La frontera de integración se materializa en tres tópicos ROS 2 que constituyen el contrato de interfaz acordado entre equipos (Tabla 15).

Tabla 15: Contrato de interfaz visión-control.

Tópico	Productor	Consumidor	Semántica
/vision_enable	Control	Visión	Indica al detector cuándo puede procesar; False al iniciar cada pick y True al terminar; latched
/ur3/target_point	Visión	Control	Posición 3D en base_link (m); solo si vision_enable=True; cola=1
/tapones/caja_asignada	Visión	Control	Índice de caja (1-6); junto al target; ambos requeridos para iniciar ciclo

Desde el punto de vista del control, el sistema de visión es una caja negra que produce pares (posición, color) cuando está habilitado. El único acoplamiento temporal relevante es el arranque: el nodo `ur3_pick_sort` publica `/vision_enable=False` al iniciarse y no lo activa hasta que el robot alcanza físicamente HOME, lo que obliga al detector a permanecer inactivo durante la maniobra inicial. El `launch.py` refleja esta dependencia retrasando el arranque del detector 25 s respecto al controlador.

2.2. Subsistema de Visión

2.2.1. Infraestructura (`lra_vision`)

El paquete `lra_vision` aporta la base sobre la que opera la clasificación: adquisición de imagen, calibración intrínseca y el modelo cinemático cámara-robot (URDF/TF).

Adquisición de imagen (C++) La captura emplea el nodo `v4l2_camera` lanzado por `camera_manager.launch.py`, que autodetecta el dispositivo. Los fotogramas se publican como `sensor_msgs/Image` y los intrínsecos como `sensor_msgs/CameraInfo` (Tabla 16). La conversión a matrices OpenCV se realiza con `cv_bridge`.

Tabla 16: Parámetros de adquisición (`camera_manager.launch.py`).

Parámetro	Valor	Significado
<code>video_device</code>	<code>/dev/video2 (auto)</code>	Dispositivo V4L2
<code>image_size</code>	<code>[640, 480]</code>	Resolución
<code>time_per_frame</code>	<code>[1, 60]</code>	Periodo $\Rightarrow$ 60 FPS
<code>pixel_format</code>	<code>rgb8</code>	Formato de píxel
<code>camera_frame_id</code>	<code>camera_optical_frame</code>	Marco óptico
<code>camera_info_url</code>	<code>file:///.../camera_info.yaml</code>	Intrínsecos

Calibración intrínseca El nodo `camera_calibrator_node` estima los parámetros intrínsecos a partir de un patrón de ajedrez (método de Zhang). Detecta las esquinas y tras acumular un mínimo de imágenes ejecuta `calibrateCamera`.

Tabla 17: Intrínsecos obtenidos (`calibration_data/camera_info.yaml`, cámara Logitech StreamCam).

f_x	f_y	c_x	c_y
630.28	630.52	319.5	239.5
k_1	k_2	k_3	RMS reproyección
0.00920	0.01708	-0,27439	1.111 px

El nodo expone un servicio `/calibrate_camera` (`lra_vision/srv/CalibrateCamera`) con acciones `start/capture/calibrate/save` y publica el progreso en `/camera/calibration/status` (`lra_vision/msg/CalibrationStatus`). La Figura 1 resume el flujo.

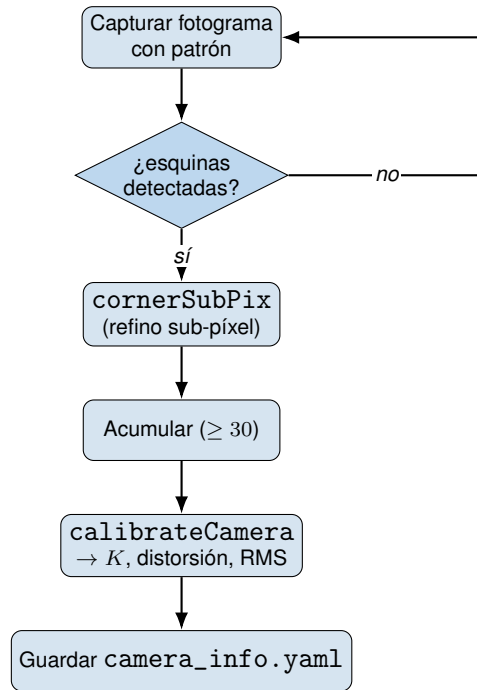


Figura 1: Flujo de calibración intrínseca.

Modelo cinemático cámara-robot (URDF/Xacro + TF) La cámara se modela en `urdf/camera.xacro` y se monta sobre el efector `tool0` del UR3 mediante uniones fijas, generando los marcos `camera_link` y `camera_optical_frame`. El `robot_state_publisher` difunde estas transformaciones en `/tf` (Figura 2); son los extrínsecos que `rec_vision` emplea para pasar de píxel a coordenadas del robot (sección 2.2.2).

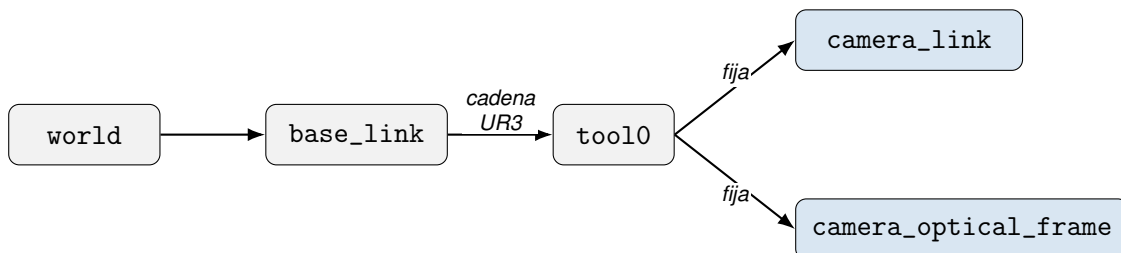


Figura 2: Árbol de transformaciones (TF). La cámara cuelga de `tool0`.

2.2.2. Clasificación de tapones (`rec_vision`)

`rec_vision` contiene el núcleo algorítmico. Se organiza en dos nodos coordinados (Figura 3) y depende de `lra_vision` para los fotogramas, el fichero `camera_info.yaml` y el TF.

Nodo 1 — `color_calibrator_node`. Se ejecuta una sola vez al arrancar: aprende, por aprendizaje no supervisado (K-Means), el color de referencia de cada caja y lo publica de forma persistente.

Nodo 2 — `vision_node / detector_tapones`. Se ejecuta en cada ciclo: detecta los tapones, estabiliza la detección, clasifica el color comparando con las referencias del Nodo 1, transforma a 3D y publica el objetivo.

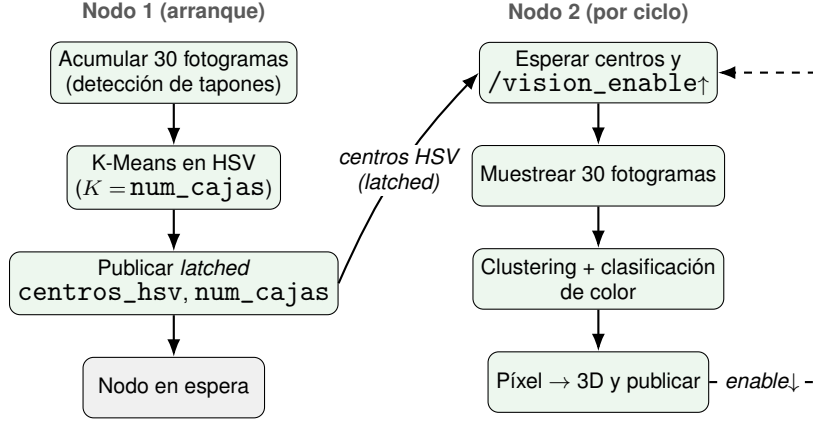


Figura 3: Arquitectura de dos nodos de `rec_vision`.

Región de interés: detección de la caja Para reducir cómputo y descartar detecciones espurias, el procesamiento se limita a una región de interés (ROI) dinámica: el contorno de mayor área de la caja de trabajo, segmentada en **HSV** (más robusto que RGB a cambios de iluminación)

$$H \in [5, 25], \quad S \in [40, 255], \quad V \in [40, 200] \quad (8)$$

Si no se detecta caja, se opera sobre el fotograma completo. Las coordenadas internas se trasladan después a la imagen global sumando el desplazamiento de la ROI.

Detección de tapones: transformada de Hough circular Dentro de la ROI, la imagen se convierte a escala de grises y se suaviza con un **filtro Gaussiano** 9×9 ($\sigma = 2$) antes de aplicar la transformada de Hough, que localiza circunferencias votando en un espacio de acumulación a partir del gradiente (Canny interno). Los parámetros se exponen como parámetros ROS para ajuste en laboratorio.

Extracción robusta de color De cada tapón detectado se extrae un color HSV representativo con tres salvaguardas: se muestrea un anillo interior entre $0,25r$ y $0,75r$ (Figura 4) para evitar el reflejo especular central y el borde; se descartan píxeles de reflejo o sombra ($V < 245 \wedge V > 25 \wedge \neg(V > 220 \wedge S < 40)$), y se agregan los píxeles con la mediana de S y V (robusta a outliers) y la media circular del matiz H , que es periódico:

$$\theta_i = H_i \cdot \frac{2\pi}{180}, \quad \bar{H} = \left(\text{atan2} \left(\frac{1}{N} \sum_i \sin \theta_i, \frac{1}{N} \sum_i \cos \theta_i \right) \cdot \frac{180}{2\pi} \right) \text{ mód } 180. \quad (9)$$

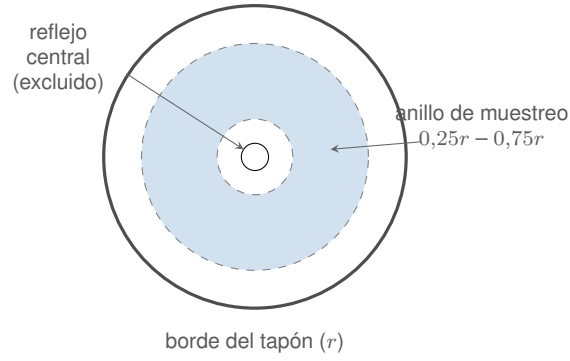


Figura 4: Máscara anular para la extracción de color.

Calibración de color por K-Means (Nodo 1) El Nodo 1 acumula los colores de unos fotogramas y aplica **K-Means** con $K = \text{num_cajas}$ sobre el espacio HSV, particionando las muestras en K grupos que minimizan la inercia intra-clúster:

$$\min_{\{\mu_1, \dots, \mu_K\}} \sum_{j=1}^K \sum_{x \in C_j} \|x - \mu_j\|^2. \quad (10)$$

Los centroides $\mu_j = (H_j, S_j, V_j)$ son el **color de referencia** de cada caja. Este enfoque no supervisado evita umbrales de color codificados: el sistema se adapta a los tapones reales presentes. Los centroides y el número de cajas se publican para que el Nodo 2 los reciba. El parámetro `num_cajas` es configurable.

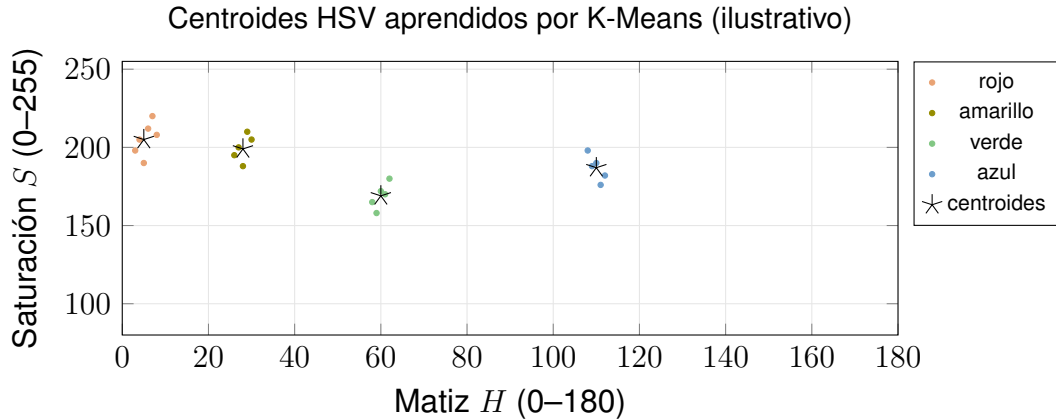


Figura 5: Muestras de color y centroides (μ_j) de cada caja en el plano matiz–saturación.

Clasificación por color (Nodo 2) El color del tapón objetivo se compara con cada centroe mediante una **distancia HSV ponderada** que respeta la circularidad del matiz y atenúa H cuando la saturación es baja. La caja asignada es la del centroe más cercano: $\text{caja} = \arg \min_j d(c_{\text{tapón}}, \mu_j) + 1$.

Estabilidad temporal multifotograma La detección fotograma a fotograma es ruidosa. Para enviar al robot un objetivo **estable**, el Nodo 2 acumula 30 fotogramas y agrupa todas las detecciones en clústeres espaciales (umbral de 10 px entre centros). El clúster ganador es el de mayor número de apariciones (el tapón cuya posición ha sido más consistente en el tiempo). Su posición se promedia y su color se resume con media circular de H y mediana de S, V (sección 2.2.2) antes de clasificar.

Transformación píxel $\rightarrow$ 3D Conocido el píxel $(\bar{u}, \bar{v})$ del objetivo, se obtiene su posición 3D en el marco `base_link` (Figura 6). Primero se corrige la distorsión y se obtiene el rayo de proyección en el marco óptico. Con la rotación R y traslación T de `camera_optical_frame` a `base_link`, el rayo en el marco del robot es $\mathbf{r}_{\text{base}} = R \mathbf{r}_{\text{cam}}$, y su intersección con el plano $z = z_t$ de la mesa da la posición del tapón.

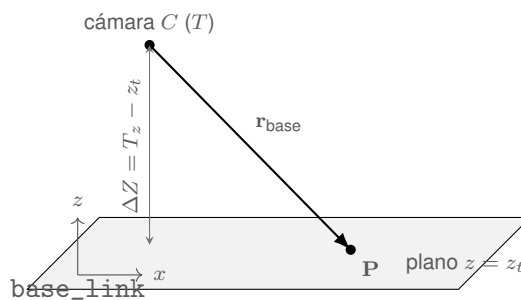


Figura 6: Geometría de la intersección rayo-plano para recuperar la profundidad.

Interbloqueo visión-robot La sincronización entre visión y manipulación se resuelve con un interbloqueo sobre el tópico `/vision_enable`. El detector arranca deshabilitado, cada flanco de subida reinicia el muestreo y produce una única detección nueva, y el flanco de bajada detiene la publicación mientras el robot maniobra (Figura 7). Así se evita enviar objetivos contradictorios durante el pick & place. La parada de emergencia de la HMI fuerza este tópico a `False`.

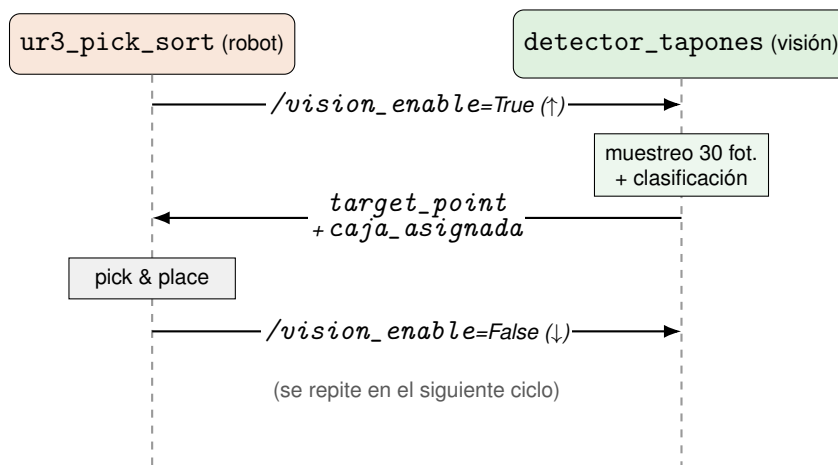


Figura 7: Diagrama de secuencia del interbloqueo visión-robot.

2.3. Interfaz Hombre-Máquina

`lra_hmi` es una interfaz de supervisión de estilo **SCADA** que actúa como centro de mando del sistema: lanza y vigila todos los subsistemas, muestra telemetría y permite la parada de emergencia. Está implementada en **PyQt5** e integra el bucle de eventos de Qt con el cliente `rclpy` en un único hilo, evitando la complejidad de un *executor* multihilo de ROS 2.

2.3.1. Arquitectura

La HMI se estructura en una ventana principal (`MainWindow`) y tres componentes desacoplados (Figura 8):

- `RosBridge`: `QObject` que posee el nodo `rclpy` y reemite cada tópico como una señal Qt. Se suscribe a la telemetría y a las cámaras con QoS *best-effort* (perfil de sensor) y al tópico *latched* `/clasificador/num_cajas` (RELIABLE + TRANSIENT_LOCAL); convierte los ángulos articulares de radianes a grados y publica `/vision_enable`.
- `ProcessManager`: lanza/para/reinicia cada subsistema como subprocesso en su propio grupo de procesos.
- `ConnectionMonitor`: `QThread` que hace *ping* ICMP periódico (cada 2 s) al robot y reporta su accesibilidad y el RTT.

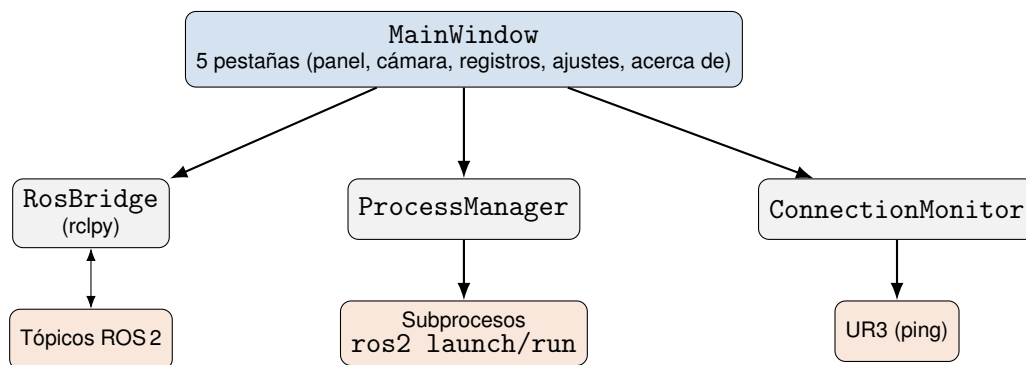


Figura 8: Componentes de la HMI.

Concurrencia. La interfaz opera en un único hilo Qt. Un `QTimer` dispara cada 50 ms una llamada a `rclpy.spin_once(timeout_sec=0.0)`, de modo que todos los *callbacks* ROS se ejecutan en el hilo de la GUI y actualizan los *widgets* sin riesgo de condiciones de carrera. Los únicos hilos concurrentes son el `ConnectionMonitor` y un hilo lector de `stdout` por cada subprocesso activo; ambos se comunican con la interfaz exclusivamente mediante señales Qt encoladas, que Qt entrega de forma segura en el hilo principal.

2.3.2. Disposición de la interfaz

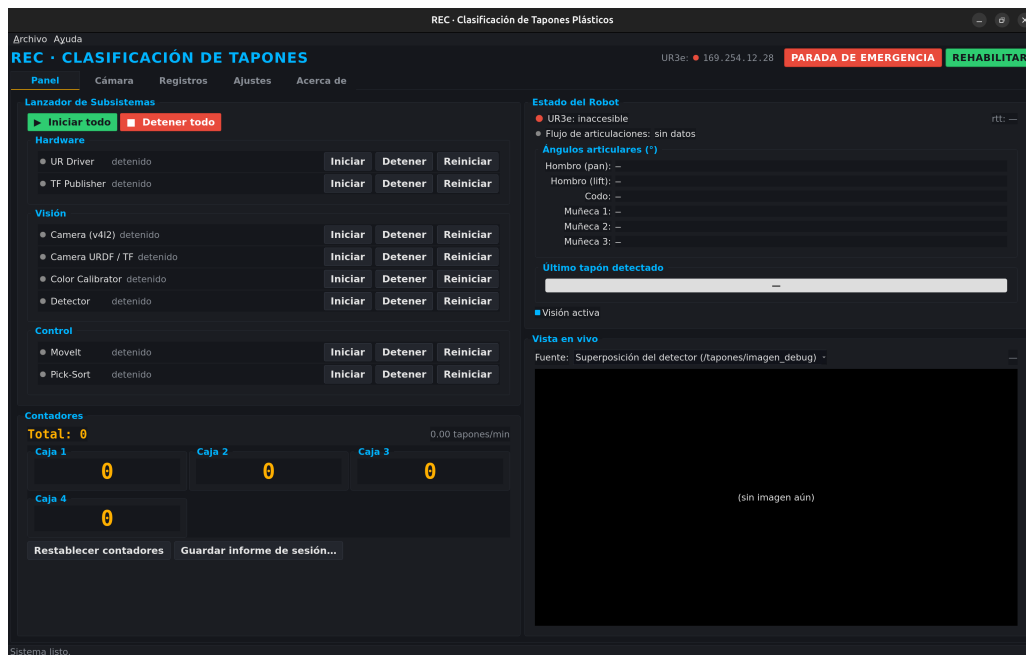


Figura 9: Captura de la HMI.

La ventana (Figura 9) tiene una barra superior siempre visible (título, LED de conexión del UR3e, IP, botón **PARADA DE EMERGENCIA** y **REHABILITAR**) y cinco pestañas (Panel, Cámara, Registros, Ajustes y Acerca de). La pestaña Panel reúne, a la izquierda, el lanzador y los contadores y, a la derecha, el estado del robot y la vista de cámara en vivo.

2.3.3. Gestión de procesos

ProcessManager lanza cada subsistema (Tabla 18) con `subprocess.Popen` y `preexec_fn=os.setsid`, creando un grupo de procesos propio. Al detenerlo envía `SIGTERM` a todo el grupo y, si no termina en 5 s, escala a `SIGKILL`; la parada de emergencia emplea `SIGKILL` directo. La salida estándar de cada grupo se vuelca en un búfer circular de 2000 líneas que alimenta las pestañas de registro. Cada subsistema sigue la máquina de estados de la Figura 10.

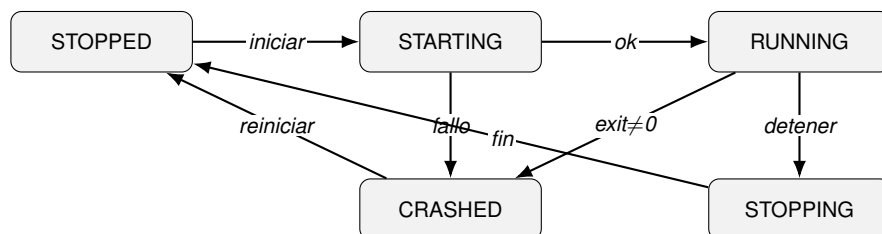


Figura 10: Máquina de estados por subsistema (ProcessManager).

La transición de STARTING a RUNNING es temporal: transcurrido el retardo de arranque propio del grupo (`startup_delay_s`), si el proceso sigue vivo se promueve a RUNNING, sin sonda de salud específica; una salida con código distinto de cero lo lleva

a CRASHED. El arranque global (**Iniciar todo**) lanza los grupos de forma secuencial respetando el retardo de cada uno.

Tabla 18: Subsistemas gestionados por la HMI (en orden de arranque).

Clave	Sección	Nodo / lanzamiento	Retardo (s)
driver	Hardware	ur_robot_driver·ur_control.launch.py	2.0
tf	Hardware	tf2_ros·static_transform_publisher	0.5
moveit	Control	ur_moveit_config·ur_moveit.launch.py	4.0
camera	Visión	v4l2_camera·v4l2_camera_node	1.0
camera_urdf	Visión	lra_vision·upload_urdf.launch.py	0.5
calibrator	Visión	rec_vision·color_calibrator_node	1.0
pick_sort	Control	ur3_vision_control·ur3_pick_sort	3.0
detector	Visión	rec_vision·detector_tapones	2.0

2.3.4. Supervisión, seguridad y configuración

- **Telemetría.** La HMI incrementa el contador de la caja recibida, muestra el total y calcula la tasa de clasificación en vivo (tapones/min desde el inicio de sesión); permite exportar un informe JSON con la duración (`duration_seconds`), el total (`total_caps`) y el conteo por caja (`per_box`). Presenta los ángulos articulares —que marca como obsoletos si no llegan datos durante más de 2 s— y el color del último tapón.
- **Seguridad. PARADA DE EMERGENCIA** publica `/vision_enable=False` y mata todos los grupos de procesos con `SIGKILL`; **REHABILITAR** vuelve a publicar `/vision_enable=True` pero *no* relanza los subsistemas, que el operador debe rearrancar desde el lanzador. `/vision_enable` es el mismo tópico *latched* del contrato visión-control (Tabla 15), por lo que el detector queda inhibido hasta la rehabilitación. El `ConnectionMonitor` señala además la pérdida de conexión con el robot.
- **Configuración.** La pestaña Ajustes edita los parámetros de cada subsistema (cámara, calibrador, detector, pick & sort) y los persiste en `~/lra_hmi.yaml`, fusionado en profundidad sobre los valores por defecto del paquete (`default_config.yaml`); al cambiar parámetros que afectan a un subsistema en ejecución, avisa de que debe reiniciarse.
- **Modo simulación.** Con la variable de entorno `LRA_HMI_SIM` activa, la HMI sustituye los grupos reales por nodos simulados de `lra_hmi_sim` y apunta el LED de conexión a `127.0.0.1`, lo que permite probar la interfaz sin robot ni cámara físicos.

2.4. Entorno de construcción y despliegue (Docker / ROS 2 Humble)

Todo el workspace se compila y ejecuta dentro de un contenedor **Docker** basado en `ros:humble` (Figura 11). Esta decisión aporta reproducibilidad (dependencias fijadas en el Dockerfile).

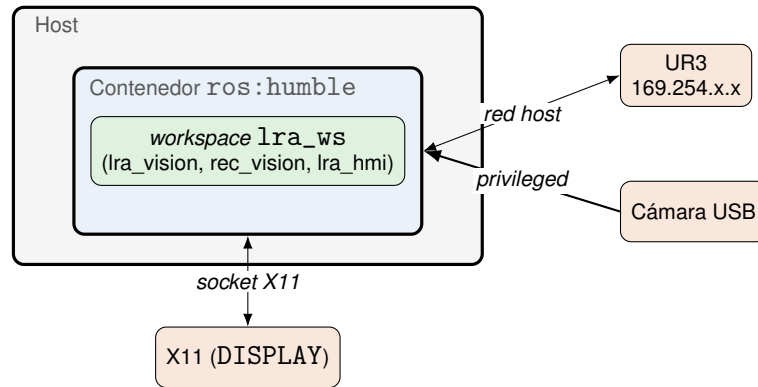


Figura 11: Despliegue en contenedor: aislamiento sobre el host, con red *host*, cámara USB y GUI por X11.

3. Pruebas de validación

3.1. Definición de las pruebas

Para verificar el correcto funcionamiento de la integración de las partes de visión y manipulación, se definieron cuatro pruebas englobadas en el paquete de trabajo WP-600 (Validación):

1. **Prueba 601: Definición de las pruebas de validación del sistema.** *Objetivo:* establecer el protocolo, el entorno físico y digital, y los criterios de aceptación para el resto de las pruebas.
2. **Prueba 602: Validación estadística de clasificación por color.** *Objetivo:* verificar la robustez del algoritmo de segmentación y clasificación cromática de los tapones bajo condiciones de ciclo continuo (detección y agarre).
3. **Prueba 603: Verificación de seguridad y exclusión de colisiones.** *Objetivo:* garantizar que el planificador de trayectorias (MoveIt) descarte rutas que comprometan la integridad del robot, detecte obstáculos virtuales y gestione correctamente los límites del espacio de trabajo.
4. **Prueba 604: Verificación de tiempos de ciclos.** *Objetivo:* medir los tiempos finales de ejecución del proceso optimizado (detección, aproximación, agarre, transporte y depósito) para comprobar si cumplen las especificaciones.

3.2. Desarrollo de las pruebas y resultados

3.2.1. Prueba 602: Clasificación por color en ciclo continuo

Procedimiento. Para comprobar la correcta clasificación por color del sistema de visión, se introdujeron en la caja 20 tapones de diferentes colores (distintos tonos de azul, verde y amarillo, además de blanco, marrón y rojos) y se predefinieron 6 cajas de clasificación. El sistema debía calibrar por color los tapones presentes, dividirlos en 6 grupos y clasificarlos en las cajas. La prueba se ejecutó un total de 20 veces.

Resultados. El sistema clasificó correctamente 18 de 20 tapones, lo que da una tasa de acierto del 90 % respecto al color. Los dos tapones restantes sí fueron depositados en otras cajas sin generar error, por lo que se obtuvo un 100 % de aciertos respecto al posicionamiento. No se observaron errores de desviación en el agarre de los tapones ni en las trayectorias.

3.2.2. Prueba 603: Seguridad y exclusión de colisiones

Procedimiento. Se realizaron tres pruebas en el entorno controlado de MoveIt y en el entorno físico:

1. *Invasión de obstáculos:* para comprobar que el sistema es capaz de detectar y esquivar obstáculos, se generó un obstáculo virtual en la trayectoria hacia una de las cajas.
2. *Coordenada prohibida:* se envió directamente una coordenada por detrás de la pared trasera.

3. *Retorno seguro*: se forzó una parada en mitad del ciclo y, desde ese punto, se probó la vuelta a la posición de HOME para verificar que el robot no invade ninguna zona restringida.

Resultados.

1. El planificador recalculó trayectorias alternativas esquivando el obstáculo virtual añadido.
2. Al enviar la coordenada por detrás de la pared, MoveIt bloqueó la ejecución del movimiento devolviendo el error esperado de trayectoria inalcanzable, impidiendo que el UR3 colisionara.
3. El retorno a HOME se realizó a través de un espacio seguro, sin invadir las zonas de los contenedores.

3.2.3. Prueba 604: Verificación de tiempos de ciclos

Procedimiento. Tras aumentar la velocidad del robot, se cronometraron 5 ciclos completos. Los tiempos por ciclo se muestran en la Tabla 19.

Tabla 19: Tiempos de ciclo registrados tras la optimización del sistema.

ID Ciclo	Detección Visión (s)	Trayectoria y Agarre (s)	Transporte y Depósito (s)	Tiempo Total (s)
Ciclo 1	< 2	10	27	39
Ciclo 2	< 2	11	26	39
Ciclo 3	< 2	10	31	43
Ciclo 4	< 2	12	27	41
Ciclo 5	< 2	11	30	43
Media	< 2	10.8	28.2	41

3.3. Análisis de resultados y validación de objetivos (requisitos)

Una vez comprobadas las pruebas de validación del paquete WP-600, se comparan los resultados obtenidos con los requisitos iniciales del proyecto:

1. **Precisión y clasificación (robustez).** La prueba 602 valida que la integración de la cámara y el software de visión es capaz de extraer las coordenadas precisas de los tapones y asociarlos a un filtro de color correcto. Con un 90 % de aciertos en color, se considera un porcentaje suficientemente alto para asumir un funcionamiento correcto y fiable ante variaciones de la posición de los tapones (con un 100 % de aciertos en posicionamiento).
2. **Seguridad (integridad).** Los resultados de la prueba 603 confirman que las herramientas empleadas (MoveIt / ROS) eliminan por completo el riesgo de colisión física, tanto por fallos de programación como por introducción de coordenadas erróneas. El robot mantiene un comportamiento seguro, respetando los límites cinemáticos, geométricos y del entorno real.

3. **Eficiencia temporal (rendimiento).** El análisis de la prueba 604 muestra un tiempo medio por ciclo de **41 segundos**. Este valor, aunque adecuado, no corresponde a una gran optimización del tiempo de ciclo.

Conclusión de la validación. Se han cumplido satisfactoriamente todos los criterios de aceptación de las pruebas del paquete WP-600. Por tanto, se alcanza con éxito el hito **H7 (Validación del sistema)** y queda confirmada la aceptación final del proyecto, con el sistema listo para el hito **H8 (Sistema final operativo)** y su demostración.

4. Planos

4.1. Planos mecánicos

[Pendiente: incluir planos mecánicos del efector / soporte de cámara.]

4.2. Planos electrónicos

[Pendiente: incluir esquemas electrónicos / conexionado de E/S.]

5. Lista de materiales

[Pendiente: incluir la lista de materiales (BOM) del sistema.]

6. Vídeos

[Pendiente: incluir enlaces a los vídeos de seguimiento y demostración final.]

7. ANEXO I: Especificación de Requisitos

[Pendiente: incluir la especificación de requisitos funcionales y técnicos.]

8. ANEXO II

[Pendiente.]